

Architectures de réseaux de neurones dans l'espace des fonctions 1-Lipschitz

Thibaut Boissin

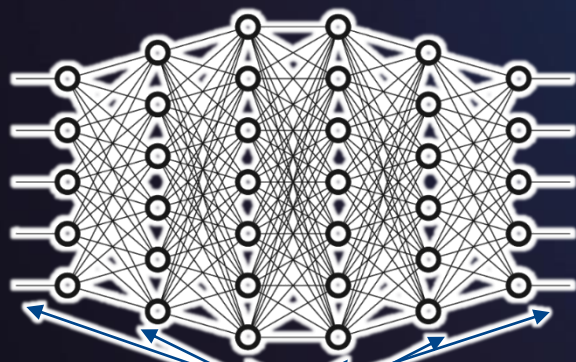
Jury: Edouard Pauwels, Christophe Garcia

Direction: Mathieu Serrurier, Franck Mamalet



1st year: Lipschitz networks

Learning l -Lipschitz functions with NN



$$f(x) = f_6 \circ f_5 \circ f_4 \circ f_3 \circ f_2 \circ f_1(x)$$

Weights

- Use **reparametrization** to constrain a layer's weights

Layer

- The layer i with constrained weights only express l_i -Lipschitz functions

Network

- Lipschitz constant is bounded by the product of each layer's Lipschitz constant

First year synthesis

- Publications

- [An adaptive orthogonal convolution scheme for efficient and flexible cnn architectures](#)
T Boissin, F Mamalet, T Fel, AM Picard, T Massena, M Serrurier
Forty-second International Conference on Machine Learning.
- [Orthogonium: A Unified, Efficient Library of Orthogonal and 1-Lipschitz Building Blocks](#)
T Boissin, F Mamalet, V Lafargue, M Serrurier
Championing Open-source DEvelopment in ML Workshop@ ICML25.
- [Efficient robust conformal prediction via lipschitz-bounded networks](#)
T Massena, T Boissin, F Mamalet, C Friedrich, M Serrurier, S Gerchinovitz
Forty-second International Conference on Machine Learning.

(2025) What's next ?

Three main lines:

- Pretraining of a Lipschitz backbone
- Exploration of applications to optimizers (Muon/Dion)
- Application to other domains (longer term)

From Lipschitz-constrained networks...

- **First year:
constraining the weights**

→ **Lipschitz networks**

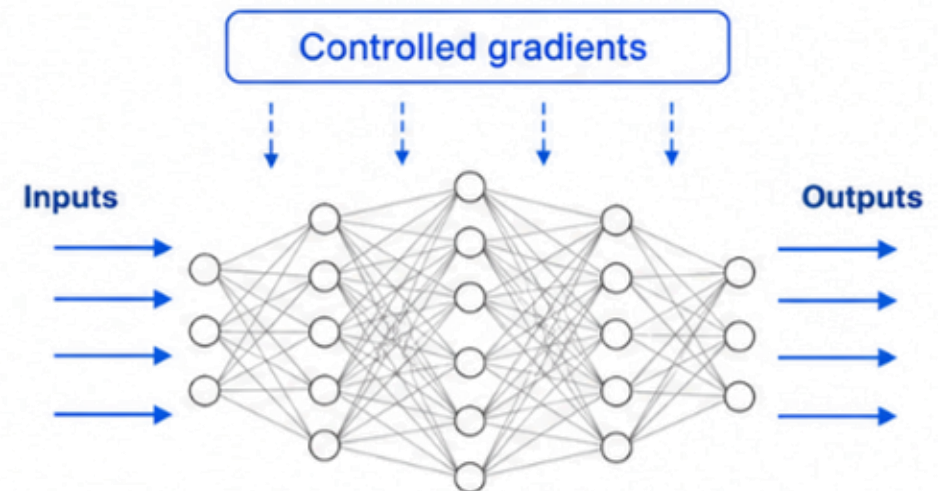


Constrain the weights to enforce
the Lipschitz property

...to gradient-constrained optimization

- **Second year:
constraining the gradients**

→ **Muon optimizer**

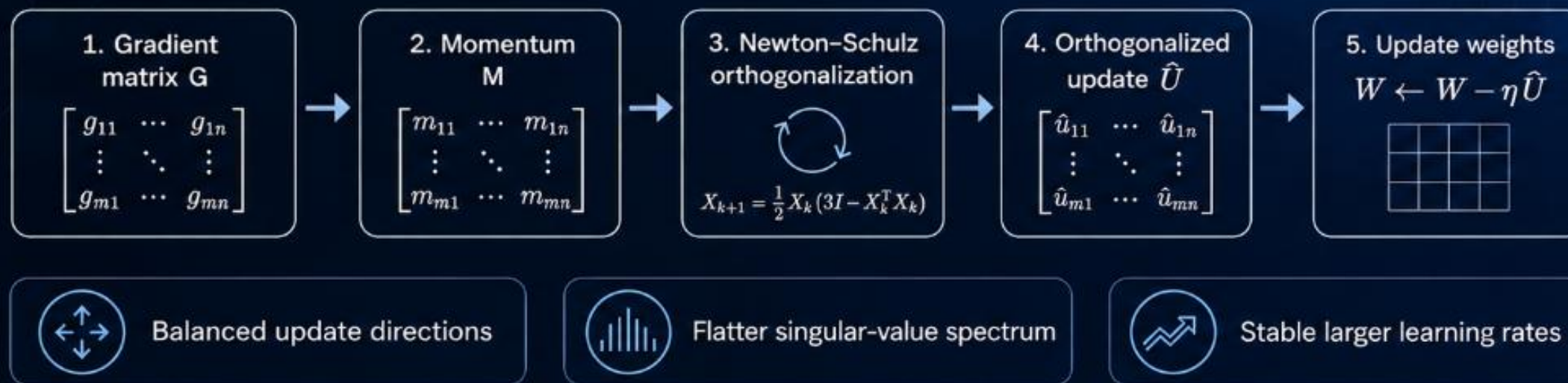


Constrain the gradients to guide
optimization with Muon

2nd year: Muon optimizer

The Muon optimizer

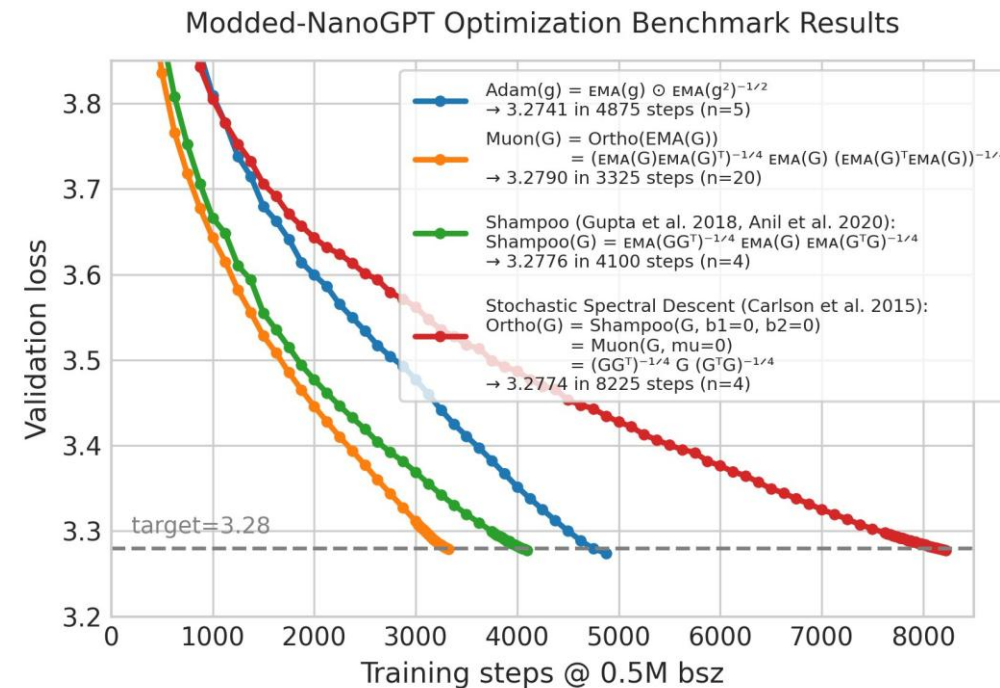
Treat hidden-layer weights as matrices, build momentum, then ortdogonalize the update before applying it.



Key idea: unlike standard optimizers, Muon refines the momentum matrix so no single direction dominates the step.

Muon is faster, but please define “fast”

- Muon needs fewer steps to achieve convergence
- But Muon adds extra computation for each step (each step is a bit slower)
- The root of its success: the **Newton-Schultz** algorithm for approximate orthogonalization.



Source: <https://x.com/kellerjordan0>

About Newton-Schulz:

- An iterative algorithm that computes a matrix polynomial approaching the *polar factor*

$$X_{k+1} = aX_k + bX_kX_k^TX_k + cX_kX_k^TX_kX_k^TX_k$$

$$\text{PolarFactor}(\mathbf{X}) = \mathbf{Q} = \mathbf{U}\mathbf{V}^T \quad \text{and} \quad \varepsilon_{\text{polar}}(Z, \mathbf{X}) = \frac{\|Z - \mathbf{Q}\|_F}{\sqrt{n}}$$

- More iterations improve step efficiency, but increase computational cost per step

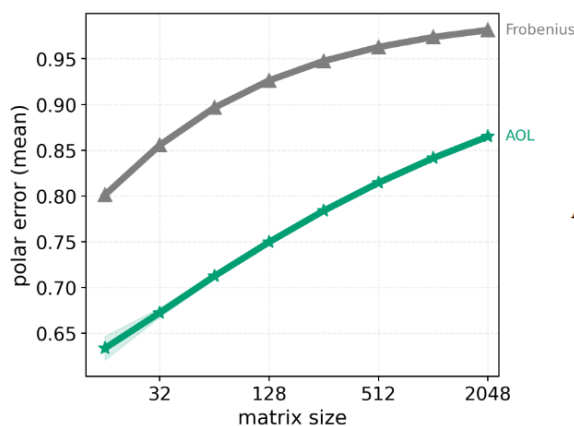
Fineweb-edu final loss	Muon
	3.34 ± 0.002 (2it)
	3.30 ± 0.002 (3it)
	3.29 ± 0.005 (4it)
	3.28 ± 0.001 (5it)

This creates a direct tension between training *speed* (time) and training *efficiency* (steps)

Turbo-Muon: Accelerating Orthogonality-Based • Optimization with Pre-Conditioning

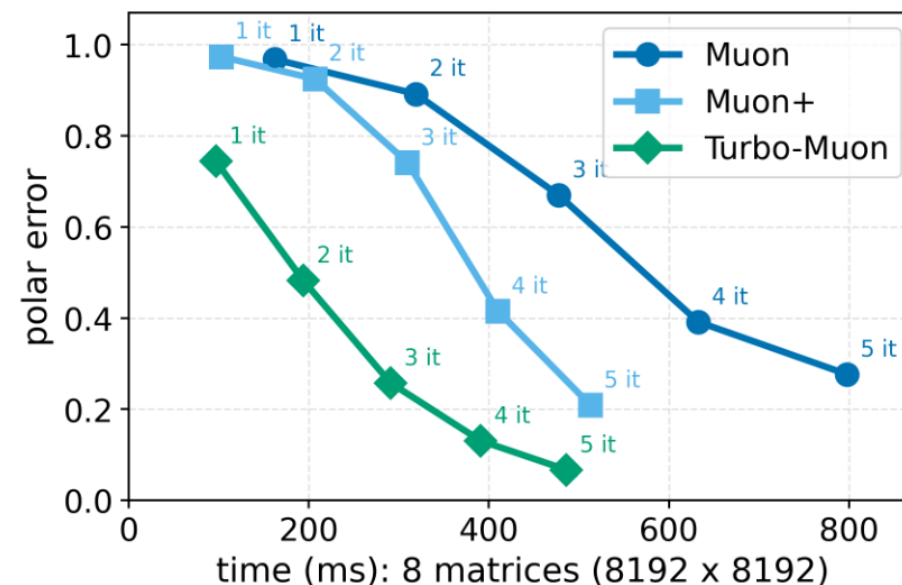
Objective

- Reduce the approximation error without slowing the algorithm
- Core idea: add a preconditioning step to accelerate convergence



(a) **Comparing pre-conditioning methods:** While normalization is primarily used for numerical stability, AOL also reduces the polar error of the initial iterate of Newton-Schulz.

$$\text{AOL}(X_0) = X_0 \cdot \text{diag} \left(\sum_j |X_0^\top X_0|_{(i,j)} \right)^{-1/2}$$



(b) **Practical implementations of orthogonalization face a trade-off between polar error and computation time.** Thanks to pre-conditioning, our method reduces the polar error of the Newton-Schulz algorithm. This improves the trade-off between convergence and runtime.

What makes our approach efficient?

How to make the preconditioning cost negligible?

What does it change in the polar error?

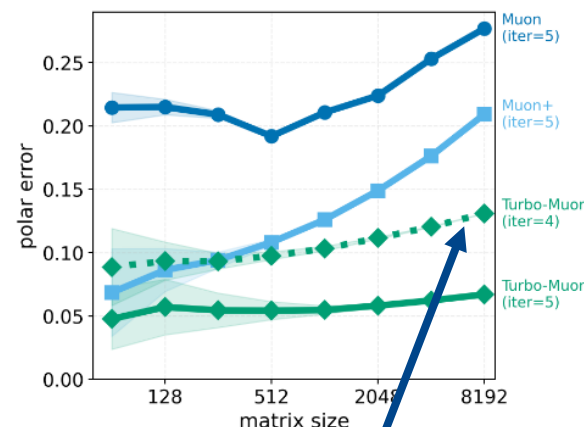
Algorithm 2 Newton-Schulz Turbo-Muon

Require: Initial point: $\mathbf{X}_0 \in \mathbb{R}^{n \times n}$, (a, b, c) NS factors

- 1: $\mathbf{A}_0 = \mathbf{X}_0^\top \mathbf{X}_0$ ▷ initial matmul
- 2: $s = (1/\sqrt{\|\mathbf{A}_{0_i}\|_1})_{i \in [0, n-1]}$ ▷ AOL vector
- 3: $\mathbf{X}_1 = \mathbf{X}_0 s$ ▷ rescaling $\mathbf{X}_1$
- 4: $\mathbf{A}_1 = s^\top \mathbf{A}_0 s$ ▷ $\mathbf{A}_1$ without recomputation
- 5: $\mathbf{B}_1 = b\mathbf{A}_1 + c\mathbf{A}_1\mathbf{A}_1$ ▷ polynomial 2
- 6: $\mathbf{X}_2 = c\mathbf{X}_1 + \mathbf{X}_1\mathbf{B}_1$ ▷ polynomial 3, $k=1$
- ...

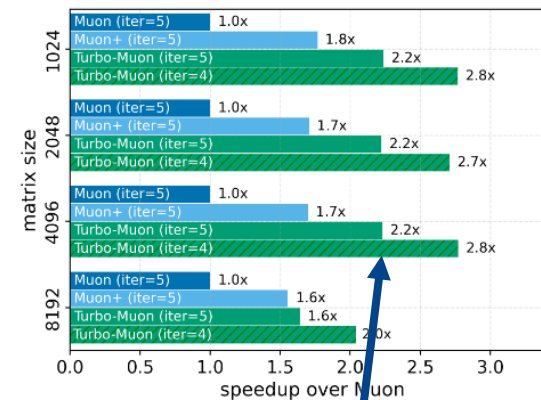
▷ Muon iteration $k+1$ (Alg. 1 step 3)

The matrix multiplication is reused



(a) **Pre-conditioning is effective for large matrices:** Applying AOL before the algorithm improves convergence speed, especially for large matrices. In this context, an iteration can be removed while still achieving improved convergence.

Preconditioning reduce the polar error enough to remove one iteration



(b) **Breakdown of our speedup:** Building atop Muon+, which includes Triton kernels and dynamic polynomial parameters (1.7x faster), our approach adds an extra Triton kernel that unlocks moderate gains (2.2x faster). Finally, removing one of the five iterations also improves runtime (2.8x faster).

Removing one iteration unlocks a speedup of Newton-Schulz

Results: same performance at a lower price

We tested on a text dataset: Fineweb-edu, a loss of 3.28 replicates the performance of GPT-2 small
We also tested on image datasets: cifar10 (50.000 images) and ImageNet (1.000.000 images)

It	Turbo-Muon	Muon+	Muon
2	$3.32 \pm 1e^{-3}$	$3.35 \pm 3e^{-3}$	$3.34 \pm 2e^{-3}$
3	$3.29 \pm 1e^{-3}$	$3.31 \pm 2e^{-3}$	$3.30 \pm 2e^{-3}$
4	$3.28 \pm 1e^{-3}$	$3.29 \pm 3e^{-3}$	$3.29 \pm 5e^{-3}$
5	$3.28 \pm 1e^{-3}$	$3.28 \pm 3e^{-3}$	$3.28 \pm 1e^{-3}$

(a) **Our approach allows the removal of one Newton-Schulz iteration:** We trained a 144M GPT model on the FineWeb dataset [27] up to the performance of GPT-2 and reported validation loss. We reused the fastest existing training script and replaced the Newton-Schulz implementation without changing any other parameters.

Dataset	Variant	Perf	Time
speed-run	Muon+	94.04%	1.359s
Cifar10	T-Muon	94.03%	1.348s (-0.8 %)
speed-run	Muon+	3.28	273.8s
nanoGPT	T-Muon	3.28	266.0s (-2.8%)
ImageNet	Muon+	69.56%	422 m
(1K)	T-Muon	69.36%	407 m (-3.6%)

(b) **End-to-end performance and runtime** for several benchmarks using Turbo-Muon with in-place replacement. Turbo-Muon (T-Muon) consistently achieves equivalent performance in less time (s: seconds, m: minutes).

**Muon is theoretically wrong for convolutions,
but empirically effective**

Why is Muon wrong for convolutions?

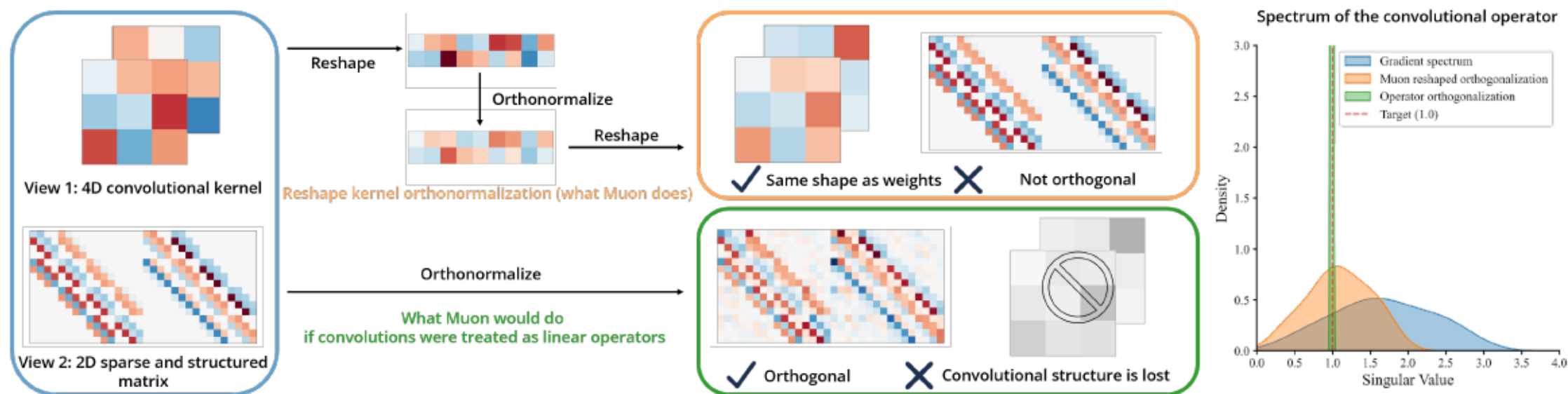


Figure 1: Problem framing for Muon on convolutions. Muon's current implementation (option 1) reshape the update and apply Orth, however this result in a non orthogonal convolution as observed on the singular value distribution. However there is no trivial solution since the direct application of Orth to the equivalent linear operator leads to a an orthogonal matrix, but the structure of the convolution is lost in the process. The 3×3 convolution with $c_i = 2, c_o = 1$ was sampled from a gaussian distribution.

A “theory-compliant” algorithm

- A “convolution aware” variant of Newton-Schulz:

Very similar to the Björck algorithm
(order 3 Newton-Schulz)

The alternating update rule becomes:

$$\Delta \tilde{K}^{(t)} = (1 + \beta_t) \Delta K^{(t)} - \beta_t \Delta K^{(t)} \circledast (\Delta K^{(t)})^\top \circledast \Delta K^{(t)} \quad (2)$$

$$\Delta K^{(t+1)} = \alpha_t \cdot \Pi_{\text{supp}}(\Delta \tilde{K}^{(t)}) \quad (3)$$

where β_t is the orthogonalization step size.

This is the Block
convolution, as in
the AOC paper!

But a better theory doesn't imply a better optimization

Two datasets:

- Cifar10 (small)
- ImageNet (large)

But also two settings:

- 1 epoch
- Longer training

This allows us to separate training speed from the overfitting issue.

Next: why the theoretically wrong implementation of Muon works so well?

Dataset (Budget)	Optimizer	Val. Acc. (%)	Train Acc. (%)	Runtime
CIFAR-10 (8 epochs)				
8 layers CNN, $bs = 2000$ 1xRTX4090	Muon - reshape	94.02 ± 0.14	97.66 ± 0.31	$3.074 \pm 0.005s$
	Muon - Conv-NS	93.98 ± 0.14	97.17 ± 0.38	$3.608 \pm 0.018s$
	SGD	91.25 ± 0.09	93.00 ± 0.35	$2.975 \pm 0.009s$
CIFAR-10 (1 epoch)				
8 layers CNN, $bs = 250$ 1xRTX4090	Muon - reshape	87.18 ± 0.002	86.46 ± 0.01	$0.509 \pm 0.003s$
	Muon - Conv-NS	87.75 ± 0.002	85.44 ± 0.02	$0.559 \pm 0.003s$
	SGD	-	-	$0.496 \pm 0.004s$
ImageNet (30 epochs)				
ResNet-50, $bs = 2048$ 4xH100	Muon - reshape	74.82	60.51	1 h 54m 06s $\pm$ 12m 51s
	Muon - Conv-NS	74.53	58.70	1 h 49m 20s $\pm$ 1m 42s
	SGD	70.25	48.03	1 h 33m 56s $\pm$ 8m 07s
ImageNet (1 epoch)				
ResNet-50, $bs = 512$ 1xH100	Muon - reshape	54.05	28.10	14m 57s $\pm$ 4m 57s
	Muon - Conv-NS	52.95	26.74	13m 13s $\pm$ 44s
	SGD	30.47	10.10	13m 00s $\pm$ 3m 46s

Table 2: **CIFAR-10 and ImageNet classification results.** Conv-NS essentially matches reshape-based Muon on CIFAR-10 AirBench94 and remains competitive on ImageNet-1k, while reshape-based Muon keeps a slight overall edge. Train and validation accuracies are shown alongside runtimes for both the full budget (8/30 epochs) and the 1-epoch setting.

Still a work in progress!

Wrap up

Publications

- Publications

- [An adaptive orthogonal convolution scheme for efficient and flexible cnn architectures](#)
T Boissin, F Mamalet, T Fel, AM Picard, T Massena, M Serrurier
Forty-second International Conference on Machine Learning.
- [Orthogonium: A Unified, Efficient Library of Orthogonal and 1-Lipschitz Building Blocks](#)
T Boissin, F Mamalet, V Lafargue, M Serrurier
Championing Open-source DEvelopment in ML Workshop@ ICML25.
- [Efficient robust conformal prediction via lipschitz-bounded networks](#)
T Massena, T Boissin, F Mamalet, C Friedrich, M Serrurier, S Gerchinovitz
Forty-second International Conference on Machine Learning.



[Back to the Baseline: Examining Baseline Effects on Explainability Metrics](#)
AM Picard, T Boissin, V Subhash, R Cadène, T Fel
arXiv preprint arXiv:2512.11433



[Turbo-Muon: Accelerating Orthogonality-Based Optimization with Pre-Conditioning](#)
T Boissin, T Massena, F Mamalet, M Serrurier
arXiv preprint arXiv:2512.04632 submitted to Neurips



[Muon is theoretically wrong for convolutions, but empirically effective.](#)
T Boissin, T Massena, F Mamalet, M Serrurier (TBD)
not published yet

- Open-Source

- Turbo-Muon is available In Jaxopt, HuggingFace kernels, and an integration in Pytorch is pending

Courses

- Others
 - PAISS summer school (25h)
 - « éthique et intégrité scientifique » (6h)
 - Others (2h)
 - Conférences: ICML poster (15h) and 6-day attendance (18h)
 - Local seminar: ANITI-days (3h)
- Teaching
 - Teaching at AIBT – Robustness module
- Award
 - Mobilit-AI (best communication and outreach award)

What's next ?

Bridging the gap between forward orthogonality (Lipschitz) and backward orthogonality (Muon):

- Orthogonal layers are invertible, and the inverse is their transpose
- Backpropagation through linear layers: the signal is backpropagated using the transposed weight matrix.
 - Back propagation computes the layer's inverse!
- Meulemans 2020: Target propagation approximate 2nd order optimization for invertible networks.
 - The specific case of orthogonal networks remains unexplored.



Dependable, Explainable & Embeddable Learning

Thank you !



• How to (not) build 1-Lipschitz backbone

The objective:

- Problem: training a robust network is time and energy-consuming:
 - Robust networks need more epochs (between 10x and 100x more)
 - Lipschitz networks need to be trained from scratch
- Idea: train a robust backbone and fine-tune on new tasks
 - Pay for the training once and use it multiple times

Preliminary: Is data all you need?

CIFAR-10

Models	Acc- uracy	Provable Accuracy $\epsilon = \frac{36}{255}$	Trainable Parameters
BCOP	72.2	58.2	2.6M
GloRo [†]	77.0	58.4	8.0M
Local-Lip-B [†]	77.4	60.7	2.3M
Cayley Large	74.6	61.4	21.0M
SOC 20	78.0	62.7	27.0M
SOC+ 20	76.3	62.6	27.0M
CPL XL [†]	78.5	64.4	236.0M
AOL Large [†]	71.6	64.0	136.0M
SLL Small [†]	71.2	62.6	41.0M
SLL Medium [†]	72.2	64.3	78.0M
SLL Large [†]	72.7	65.0	118.0M
SLL X-Large [†]	73.3	65.8	236.0M
Li-Resnets ^{†*}	82.1	70.1	49.0M
Li-Resnets++ ^{†*}	87.0	78.1	49.0M
AOC accurate	91.5	00.0	2.3M
AOC robust	74.0	64.3	41.3M

Takes 24h to train

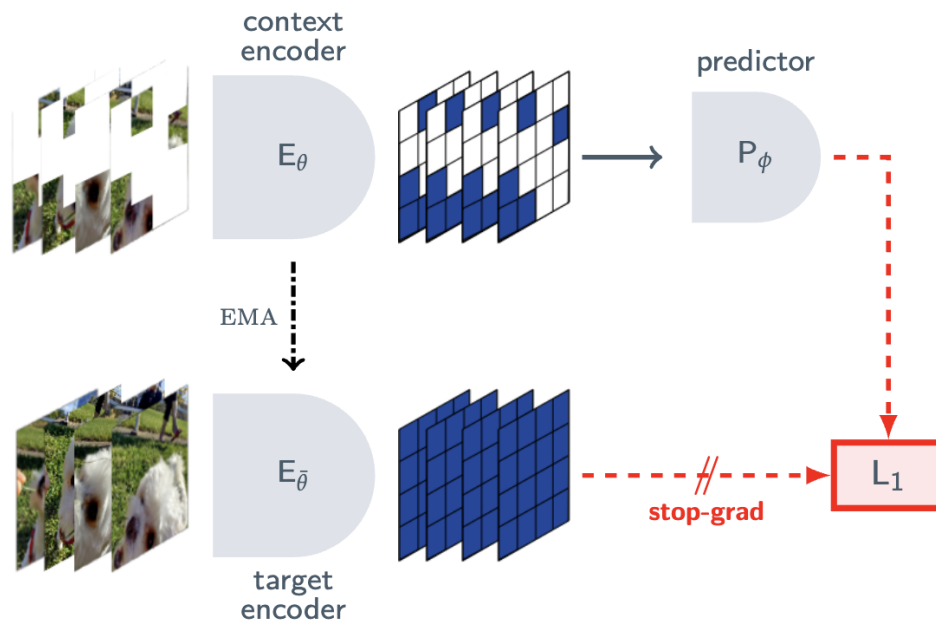


Takes 2h to train

Multiply dataset size
by 1000x

- Problem: Imagenet1k has 1Millions images => we would need 1Trillion images !

Idea: Self-supervision



- Source:todo

- Unlocks larger datasets:
 - Laion-COCO: 600M images
- Lipschitz are compliant with existing SSL algorithm:
 - LeJepa seems like a promising choice

Results:

- Cifar10: promising results
 - >50% accuracy without seeing a single Cifar10 label!
- Imagenet/Laion: disappointing results:
 - 300M LAION images: around 20% accuracy on imagenet
 - 300M LAION x 300 Imagenet Epochs: 37%
 - Models with 256M params, trained during 24h, on 8xH100
 - Almost no certifiable robustness

